WPT Day 10 Morning

Fri 30 Oct 26 · Cleaning up the Express routers with a single prefix in `app.use()`, then an introduction to TypeScript: why it exists, `tsc`, `tsx` and `@types/node`, `devDependencies`, `tsconfig.json`, typed variables / functions / arrays, interfaces, type aliases, and union (`|`) vs intersection (`&`) types

---

Table of Contents

1. Express Router Prefix Optimization
2. Registering Middleware Once
3. Why TypeScript Exists
4. TypeScript Is Development-Only
5. The TypeScript Compiler tsc
6. Project Setup: Installing TypeScript
7. Dependencies vs devDependencies
8. tsconfig.json: Configuring the Compiler
9. Strict Type Checking Options
10. Typing Variables, Functions and Arrays
11. Interfaces: Defining Object Shapes
12. Type Aliases: Custom Data Types
13. Union and Intersection Types
14. interface vs type and Naming Conventions
15. Combined Worked Example
16. Quick Revision Checklist
17. Practice Exercises
18. Practice Answers

Express Router Prefix Optimization

The problem

The Express application built on 29 October (the secured CRUD application) has two routers:

- `productRouter` — every route in it was written with the prefix `/product`.
- `adminRouter` — every route in it was written with the prefix `/admin`.

That is repetitive and error-prone. If the prefix ever has to change (say `/product` becomes `/products`), **every single route definition** in the router file has to be edited, and missing one produces a bug that only shows up when somebody calls that one URL.

Before — the prefix is repeated on every route:

```
// product-router.js (before)
router.get('/product', verifyToken, getAllProducts);
router.get('/product/:id', verifyToken, getProductById);
router.post('/product', verifyToken, createProduct);
router.put('/product/:id', verifyToken, updateProduct);
```

```
router.delete('/product/:id', verifyToken, deleteProduct);

// admin-router.js (before)
router.get('/admin', getAllAdmins);
router.post('/admin/login', adminLogin);
```

The mechanism: move the prefix into `app.use()`

`app.use()` accepts a **path as its first argument**. When the prefix is given there, Express forwards every request whose URL begins with that prefix to the router, and the router only has to describe what comes *after* the prefix.

```
Request:  GET /product/7

index.js                        product-router.js
─────────────────────────       ──────────────────────────
app.use('/product', ...)  ──►   router.get('/:id', ...)
  matches the prefix              sees only "/7"  (the suffix)
  "/product"                      so the route is just "/:id"
```

Background: when a router is mounted at a path, Express strips that mount path from the URL before the router sees it. Inside the router the leftover part (`/7`) is what route patterns are matched against; the stripped part is still available as `request.baseUrl` . That is why the router must **not** repeat the prefix — if it did, the real URL would become `/product/product/7` .

After — the prefix is defined once in `index.js` :

```
// index.js (after)
import productRouter from './routers/product-router.js';
import adminRouter from './routers/admin-router.js';
import { verifyToken } from './middlewares/verify-token.js';

app.use('/product', verifyToken, productRouter);
app.use('/admin', adminRouter);
```

```
// product-router.js (after) - only suffix paths remain
router.get('/', getAllProducts);
router.get('/:id', getProductById);
router.post('/', createProduct);
router.put('/:id', updateProduct);
router.delete('/:id', deleteProduct);

// admin-router.js (after)
router.get('/', getAllAdmins);
router.post('/login', adminLogin);
```

Before vs after, URL by URL

Public URL	Before (route inside router)	After (route inside router)
`GET /product`	`router.get('/product', ...)`	`router.get('/', ...)`
`GET /product/5`	`router.get('/product/:id', ...)`	`router.get('/:id', ...)`
`POST /admin/login`	`router.post('/admin/login', ...)`	`router.post('/login', ...)`

The **public URLs do not change at all** — Postman collections and front-end code keep working. Only where the prefix is written changes.

> **Exam:** Prefix once in `app.use('/prefix', router)`; the router file defines only the **suffix** (`'/'`, `'/:id'`, `'/login'`). To rename a prefix later you edit one line.

Registering Middleware Once

The problem

On Day 9 the `verifyToken` middleware was written on **every** individual product route (`router.get('/product', verifyToken, getAllProducts)` and so on). Every product route needs authentication, so the same word was typed five times — and forgetting it on one route would leave that route silently open.

The mechanism

`app.use()` takes the path, then **any number of handlers**. Handlers run left to right, exactly like the chain on a single route. So the middleware can be placed in `app.use()` next to the router:

```
// verifyToken applied once – covers all /product routes
app.use('/product', verifyToken, productRouter);
```

Order of execution for `DELETE /product/3`:

```
request ──► app.use('/product', ...) prefix matches
              |
              v
          verifyToken            token missing / invalid ──► 401, STOP
              |  next()
              v
          productRouter          router.delete('/:id', deleteProduct)
              |
```

```
                v
deleteProduct          real work, sends the response
```

What stays unguarded, on purpose

```
app.use('/admin', adminRouter);    // NO verifyToken here
```

The admin router holds the **login** route (`POST /admin/login`). From Day 9: register and login must stay open, otherwise nobody could ever obtain a token. So `verifyToken` is attached to `/product` only.

The general principle

> The same principle applies to any future entity (e.g. `/orders` , `/cart`). Define the prefix and shared middleware once in `index.js` ; keep only the unique suffix in the router.

Concern	Where it lives
Prefix (`/product`)	`app.use()` in `index.js`
Shared middleware (`verifyToken`)	`app.use()` in `index.js`
Unique suffix (`/` , `/:id`)	the router file
Business logic	the controller

> **Pitfall:** Do not put `verifyToken` in front of a router that contains the login route. `app.use('/admin', verifyToken, adminRouter)` would demand a token to *get* a token — and login could never succeed.

Why TypeScript Exists

JavaScript's core problem: no type safety

JavaScript is a **dynamically typed** language. In practice that means:

- You can declare a variable but **cannot give it a fixed data type**.
- A variable can **silently change its type at runtime** with no error.

- Function **arguments have no enforced types** — you can pass a string where a number is expected.
- Functions have **no declared return type** — they can return anything, or nothing.
- JSON objects have **no enforced shape** — required keys can be missing and extra keys can appear.

```
// JavaScript — no type safety
let x = 5;
x = "hello";          // No error! x silently becomes a string

function sum(a, b) {
  return a + b;
}
sum(3, "eight");      // No error! Returns "3eight" — a silent bug
```

Why `sum(3, "eight")` returns `"3eight"`: `+` adds two numbers, but if either side is a string it **concatenates** instead; JavaScript converts `3` to `"3"` and glues the strings together. Nothing complains.

> These issues do not throw errors during development. They surface as bugs at **runtime**, often in production, making them expensive and difficult to trace.

TypeScript: JavaScript with added data types

TypeScript is **not** a completely new language. It is JavaScript with an added layer of **type annotations**. Every JavaScript feature — `let`, `const`, `var`, `console.log`, promises, arrow functions, classes, modules — works exactly the same in TypeScript. The only addition is that you can now declare data types for:

- variables,
- function arguments,
- function return types,
- object shapes.

Point	Meaning
TypeScript = JavaScript + data types	a superset of JS
All JS syntax is valid TS	existing JS knowledge carries over fully
Type safety is enforced at **development time only**	on the developer's machine, never in production

Point	Meaning
It catches bugs **before the code ever runs**	errors appear in the editor / compiler

Background: "Statically typed" means types are checked **before** the program runs (compile time); "dynamically typed" means they are checked only while it runs. TypeScript bolts a static check on top of a dynamic language.

> **Exam:** TypeScript = JavaScript + data types. JS is dynamically typed (bugs appear at runtime); TS adds type checking at development time (bugs appear before the code runs).

TypeScript Is Development-Only

Never deploy TypeScript

TypeScript must **not** be deployed to any environment — testing, staging, UAT or production. The reasons:

- If you deploy `.ts` files directly, the runtime has to compile them to JavaScript **on every single execution** — for every request, every function call.
- That adds pointless compilation overhead to every execution cycle.

Node.js (and browsers) only understand JavaScript, so the correct workflow is:

write in TS → compile to JS → deploy the JS

```
Developer Machine          Build Step              Server
─────────────────          ──────────              ──────
Write .ts files     ⟶    tsc compiles to .js  ⟶   Deploy .js only
(type-safe, safe)         (resultant JavaScript)   (fast execution)
```

TypeScript's role **ends at the build step**. The deployed artefact is always plain JavaScript.

Where	What exists	Types checked?
Developer machine	`.ts` files in `src/`	Yes
Build step	`tsc` converts `.ts` to `.js`	Yes — errors stop the build
Server	`.js` files from `dist/`	No — types are gone, only JS runs

> **Exam:** Never deploy `.ts`. Always compile first, deploy the compiled JavaScript (from `dist/`).

The TypeScript Compiler tsc

TypeScript ships with its own compiler called **`tsc`**. The lecture's analogy: Java ships with **`javac`** inside the JDK.

Language	Compiler
Java	`javac`
TypeScript	`tsc`

The `typescript` npm package provides **two** things:

1. **`tsc`** — the compiler that converts `.ts` files into `.js` files.
2. **The type-checker engine** — reads your code in real time and raises type errors as you write.

> JavaScript has no compiler. TypeScript does. This compiler is what makes the entire type-safety workflow possible.

Project Setup: Installing TypeScript

Step 1: initialise the project

```
npm init -y
```

This creates `package.json` with default values. Then switch it to ES Modules by adding one line:

```
{
  "type": "module"
}
```

Step 2: install the three core dev dependencies

```
npm i typescript tsx @types/node --save-dev
```

The `--save-dev` flag records them under `devDependencies` in `package.json`: they are installed on developer machines, not on production servers.

What each dependency does

1. `typescript`

- The core TypeScript package.
- Installs the `tsc` compiler.
- Installs the type-checker engine.
- Without it, nothing TypeScript-related works.

2. `tsx`

- A TypeScript **execution engine for the developer machine only**.
- Internally compiles your `.ts` code and runs it automatically — you do not compile manually after every change.
- Think of it like **nodemon but for TypeScript**: save a file, it recompiles and reruns.
- **Never use `tsx` in production.** It compiles on every execution, which wastes time. In production you deploy pre-compiled JS.
- It is optional — if you opt out, you must run `npx tsc` and then `node` by hand after every change.

3. `@types/node`

- When Node.js was installed, its built-in modules (`fs`, `http`, `os`, `path`, …) came as plain JavaScript — with **no type definitions**.
- TypeScript needs type definitions to know what functions exist in those modules and what types they accept and return.
- `@types/node` supplies those definitions for all Node.js built-in modules.
- Without it, TypeScript would not know what `fs.readFile` or `http.createServer` looks like.

Package	Role	Dev-only?
`typescript`	`tsc` compiler + type checker	Yes
`tsx`	compiles and runs `.ts` automatically (like nodemon)	Yes — never in production

Package	Role	Dev-only?
`@types/node`	type definitions for Node built-ins	Yes

> **Exam:** `tsc` = compile; `tsx` = compile-and-run on the dev machine; `@types/node` = type definitions for Node's built-in modules.

Dependencies vs devDependencies

The two lists in `package.json`

```
{
  "dependencies": {
    "express": "^4.18.2",
    "mysql2": "^3.6.0",
    "bcrypt": "^5.1.0",
    "jsonwebtoken": "^9.0.0"
  },
  "devDependencies": {
    "typescript": "^5.3.0",
    "tsx": "^4.6.0",
    "@types/node": "^20.10.0"
  }
}
```

List	Meaning	Examples here
`dependencies`	needed to **run** the app — on every environment (local, testing, staging, production)	express, mysql2, bcrypt, jsonwebtoken
`devDependencies`	needed **only on the developer's machine** while developing	typescript, tsx, @types/node

Test for deciding: "*Without this package, can the compiled code still run?*" If no (Express, mysql2…) → `dependencies`. If yes (TypeScript, once the JS exists) → `devDependencies`.

Why the separation matters

The `node_modules` folder is notoriously heavy — it can contain tens of thousands of files. Every dependency deployed to a server occupies real disk space, and on cloud servers **you pay per byte**. DevOps engineers write deployment scripts (often with Docker) that install **only** `dependencies`:

```
# Production install — skips devDependencies
npm ci --omit=dev
```

Another dev-only example: ESLint

ESLint analyses your code for quality and style violations. It reads and checks source code, but has nothing to do with how the application *runs*. It is a dev dependency and must never go to production.

> **Pitfall:** Installing TypeScript without `--save-dev` puts it under `dependencies`, so the production server downloads a compiler it never uses.

---

tsconfig.json: Configuring the Compiler

Purpose

`tsconfig.json` tells the TypeScript compiler **how to behave**. It controls:

- which files and folders to include or exclude,
- where to read TypeScript source from (`rootDir`),
- where to write compiled JavaScript (`outDir`),
- which ECMAScript standard to target,
- how strict the type checking should be.

Why `tsc --init` fails — and the `npx` fix

Because TypeScript was installed with `--save-dev`, it lives **locally** in the project's `node_modules`, not globally on the system. When you type `tsc` in the terminal, the shell searches the **global** locations (where Node itself is installed). `tsc` is not there, so:

```
tsc : The term 'tsc' is not recognized as the name of a cmdlet ...
```

`npx` forces the lookup into the **current project's** `node_modules`:

```
# Wrong — looks globally, fails
tsc --init

# Correct — looks inside project node_modules
npx tsc --init
```

`npx tsc --init` generates a `tsconfig.json` at the project root.

The sample `tsconfig.json`

```
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "rootDir": "./src",
    "outDir": "./dist",
    "sourceMap": true,
    "noImplicitReturns": true,
    "noImplicitOverride": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true
  },
  "include": ["src"],
  "exclude": ["node_modules", "dist"]
}
```

Key settings explained

`rootDir: "./src"` — tells the compiler where your TypeScript source lives. It also controls whether the compiled output **preserves your folder structure**.

Without `rootDir`, with a project like this:

```
myproject/
  tsconfig.json
  core/
    a.ts
    b.ts
    sub/
      c.ts
```

the compiler might output (the `core/` folder is lost):

```
dist/
  a.js
  b.js
  sub/
    c.js        (core/ folder is lost)
```

With `rootDir: "./core"` (or `"./src"`), the structure is preserved:

```
dist/
  core/
    a.js
    b.js
    sub/
      c.js      (folder structure preserved)
```

`outDir: "./dist"` — `dist` stands for *distributable*. All compiled JavaScript is written here. This is the folder you **deploy**. The `src` folder (TypeScript) never goes to production.

`module` and `target` — tell the compiler which ECMAScript standard to follow. `ESNext` means the latest. Leave these at their defaults unless you have a specific reason.

`include: ["src"]` — compile only files inside `src`. The recommended explicit pattern is `"src/**/*"` (all files and subfolders recursively); either form works.

`exclude: ["node_modules", "dist"]` — stops the compiler from trying to compile third-party packages or its own previous output. Without excluding `node_modules`, it would attempt to compile thousands of library files — never what you want.

Option	Role
`rootDir`	where `.ts` source lives; keeps folder structure in output
`outDir`	where compiled `.js` goes (`dist`); the deployable folder
`target` / `module`	ECMAScript standard; leave `ESNext`
`include`	what to compile (`src`)
`exclude`	what never to compile (`node_modules` , `dist`)
`sourceMap`	generates `.map` files linking compiled JS back to the original TS (useful for debugging)

> **Exam:** `npx tsc --init` creates `tsconfig.json`. Plain `tsc` fails because TypeScript is installed locally, not globally. `rootDir` = input, `outDir` = output (`dist`).

Strict Type Checking Options

These flags make the compiler stricter. Each can be toggled `true` or `false` .

Setting	What it does
`noUncheckedIndexedAccess`	Warns when you access an array index that may not exist — e.g. an array of 5 elements read at index 10
`noUnusedLocals`	Error if a local variable is declared but never used
`noUnusedParameters`	Error if a function parameter is declared but never used
`noFallthroughCasesInSwitch`	Error if a `switch` case is missing a `break` (and would fall through)
`noImplicitReturns`	Error if a function does not return a value on **all** code paths
`noImplicitOverride`	Error if a subclass method overrides a parent method without the `override` keyword
`exactOptionalPropertyTypes`	If an optional property is absent, TypeScript supplies `undefined` automatically — you must not write `undefined` as the value yourself (details in the Interfaces section)

Small examples:

```
// noUncheckedIndexedAccess
const arr: number[] = [1, 2, 3, 4, 5];
const v: number = arr[10];
// Error: Type 'number | undefined' is not assignable to type 'number'.

// noImplicitReturns
function check(n: number) {
  if (n > 0) {
    return "positive";
  }
  // Error: Not all code paths return a value.
}
```

> **Exam:** The `no…` flags turn silent mistakes (unused variables, missing `break`, missing return, out-of-range index) into compile-time errors. All are on/off switches in `tsconfig.json`.

Typing Variables, Functions and Arrays

All source files now use the `.ts` extension and live in `src/`:

```
src/
  index.ts
  product-controller.ts
  product-router.ts
  verify-token.ts
```

Typing variables

The syntax is: `let variableName: dataType = value`.

```ts
// Correct — number type assigned
let x: number = 5;
console.log(x);

// Error — assigning a string to a number variable
x = "hello";   // Type 'string' is not assignable to type 'number'

let y: string = "world";
y = 6;         // Type 'number' is not assignable to type 'string'
```

Contrast with the JavaScript example at the start of the notes: `let x = 5; x = "hello";` was silently accepted. With `: number` the same line is now a compile-time error.

Typing functions

```ts
// Arguments typed as number, return type declared as number
function sum(a: number, b: number): number {
  return a + b;
}

// Return type is void — function returns nothing
function show(): void {
  console.log("Hello");
}

// Calling sum correctly
const rs: number = sum(3, 8);
console.log(rs);        // 11

// Calling sum with the wrong argument type
sum(3, "eight");        // Argument of type 'string' is not assignable to parameter of type 'number'
```

Part	Where it is written	Example
Argument types	inside the parentheses	`a: number`

Part	Where it is written	Example
Return type	after the closing parenthesis	`): number`
`void`	return type of a function that returns nothing	`): void`

The silent bug `sum(3, "eight")` → `"3eight"` from JavaScript is now caught before the program ever runs.

Typing arrays

```
// Array that only accepts numbers
let arr: number[] = [10, 20, 30];

arr.push("hello");   // Argument of type 'string' is not assignable to parameter of type 'number'
```

The syntax for a typed array is `dataType[]`. Any type works, including custom types like `IProduct[]` (next section).

Declaration	Allowed values
`let n: number`	numbers only
`let s: string`	strings only
`let a: number[]`	an array containing only numbers
`function f(): void`	returns nothing

Interfaces: Defining Object Shapes

The problem without interfaces

In plain JavaScript any object can have any keys. Nothing stops a developer from creating a product with missing required fields or unexpected extras. TypeScript solves this with **interfaces**.

Defining an interface

```
interface IProduct {
  id: number;
  name: string;
  price: number;
```

```
  quantity: number;
}
```

Any object declared as type `IProduct` must contain **exactly** these four properties with the correct types.

```
const product: IProduct = {
  id: 101,
  name: "Keyboard",
  price: 1500,
  quantity: 10
};
```

Missing property error

```
const product: IProduct = {
  id: 101,
  name: "Keyboard",
  price: 1500
  // quantity is missing
};
// Error: Property 'quantity' is missing in type '{ id: number; name: string; price:
number; }'
//        but required in type 'IProduct'
```

Array of an interface type

```
const productsList: IProduct[] = [
  { id: 101, name: "Keyboard", price: 1500, quantity: 10 },
  { id: 102, name: "Mouse", price: 800, quantity: 25 },
  { id: 103, name: "Headphone", price: 2000, quantity: 5 },
  // Adding an object with unknown properties:
  { id: 104, email: "test@test.com", phone: "9999999999" }
  // Error: Object literal may only specify known properties,
  //        and 'email' does not exist in type 'IProduct'
];
```

So the interface blocks **both** mistakes: a **missing** required key and an **extra** unknown key.

Optional properties (`?`)

Some properties may or may not be present. Mark them with `?`:

```
interface IProduct {
  id: number;
  name: string;
```

```
  price: number;
  quantity: number;
  description?: string; // optional
}
```

- Objects **without** `description` are valid — no error.
- Reading `.description` on an object that lacks it gives `undefined`.

`exactOptionalPropertyTypes` and explicit `undefined`

With `exactOptionalPropertyTypes: true`, the rule is: if an optional property is absent, TypeScript returns `undefined` for you automatically. You must **not** assign `undefined` to it yourself:

```
const product: IProduct = {
  id: 101,
  name: "Keyboard",
  price: 1500,
  quantity: 10,
  description: undefined   // Error when exactOptionalPropertyTypes is true
  // Type 'undefined' is not assignable to type 'string'
};
```

If you set `exactOptionalPropertyTypes: false` in `tsconfig.json`, the error **disappears** — the compiler becomes lenient. This shows how `tsconfig.json` directly controls compiler behaviour.

`description` situation	Allowed with `exactOptionalPropertyTypes: true` ?
key left out entirely	Yes — reading it gives `undefined`
`description: "Wireless"`	Yes
`description: undefined` written explicitly	**No** — error

Read-only properties

Prevent a property from being reassigned after creation:

```
interface IProduct {
  id: number;
  name: string;
  readonly price: number; // cannot be changed after creation
  quantity: number;
}

const product: IProduct = {
```

```
  id: 101,
  name: "Keyboard",
  price: 1500,
  quantity: 10
};

// Allowed – name is not read-only
product.name = "Wireless Keyboard";

// Not allowed – price is read-only
product.price = 45;   // Cannot assign to 'price' because it is a read-only property
```

Use `readonly` when a property should be set once at creation and never mutated — for example a database-generated ID or a fixed configuration value. You can make **all** properties `readonly` if the whole object should be immutable.

Modifier	Syntax	Effect
optional	`description?: string`	may be absent
read-only	`readonly price: number`	cannot be reassigned after creation

> **Exam:** An interface defines the shape of an object (or class). It rejects missing required keys, unknown extra keys and wrong value types. `?` = optional, `readonly` = set once.

Type Aliases: Custom Data Types

When to use `type` instead of `interface`

`interface` is ideal for the shape of objects and classes. Sometimes, however, you need a custom data type that is **not** an object shape — for example, a variable that can hold only one of a fixed set of strings. For that, use the `type` keyword.

Basic type alias

```
type TBook = {
  id: number;
  name: string;
  ISBN: string;
  price: number;
};
```

```
const book: TBook = {
  id: 1,
  name: "My Book",
  ISBN: "978-3-16-148410-0",
  price: 499
};
```

Union of literal values — the key advantage of `type`

A `status` field should only ever be `"approved"`, `"pending"` or `"rejected"`. Typed as `string`, any string slips through:

```
let status: string = "approved";
status = "banana";   // No error — but this is wrong!
```

With a type alias made of a **union of literal values**, only those exact values are allowed:

```
type TStatus = "approved" | "pending" | "rejected";

let myStatus: TStatus = "approved";   // OK
myStatus = "pending";                 // OK
myStatus = "rejected";                // OK
myStatus = "hello";                   // Error: Type '"hello"' is not assignable to
type 'TStatus'
```

This is something an interface **cannot** do. Interfaces define object shapes; they cannot restrict a variable to a set of literal values.

Using a type alias inside an interface

The two can be combined — an interface property may use a type alias:

```
type TStatus = "approved" | "pending" | "rejected";

interface IUser {
  id: number;
  name: string;
  email: string;
  status: TStatus;   // only "approved" | "pending" | "rejected" allowed
}

const user: IUser = {
  id: 1,
  name: "Shivam",
  email: "shivam@example.com",
```

```
  status: "approved"   // OK
};
```

> **Exam:** Use `type` when you need something that is not an object shape — especially a union of literal values such as `"approved" | "pending" | "rejected"`. An interface cannot do that.

Union and Intersection Types

Union types: either / or (|)

A union means a variable can hold **one of several types**. Use the `|` (pipe) operator:

```
let z: number | undefined;

z = 5;           // OK
z = undefined;   // OK
z = "hello";     // Error: Type 'string' is not assignable to type 'number | undefined'
```

A reusable alias avoids repeating the union everywhere:

```
type TUndefinedNumber = number | undefined;

let z: TUndefinedNumber = 8;   // OK
z = undefined;                 // OK
```

Intersection types: both / and (&)

An intersection means a value must satisfy **all** of the combined types at once. Use the `&` operator. It builds composite types from reusable building blocks.

```
type TPerson = {
  name: string;
  phone: string;
  email: string;
};

// TEmployee has all TPerson properties AND salary
type TEmployee = TPerson & {
  salary: number;
};
```

```ts
// TStudent has all TPerson properties AND marks
type TStudent = TPerson & {
  marks: number;
};
```

Result:

- every `TEmployee` object must have `name`, `phone`, `email`, `salary`;
- every `TStudent` object must have `name`, `phone`, `email`, `marks`.

The shared properties (`name`, `phone`, `email`) are written **once** in `TPerson` and reused — no repetition.

```
TPerson  { name, phone, email }
   |
   +--&-- { salary }  =  TEmployee { name, phone, email, salary }
   |
   +--&-- { marks }   =  TStudent  { name, phone, email, marks }
```

Union `|` vs intersection `&` — the key difference

```ts
// Union: either TPerson OR { salary } — not both required
type TBad = TPerson | { salary: number };
// An object only needs to satisfy ONE side

// Intersection: TPerson AND { salary } — both required
type TGood = TPerson & { salary: number };
// An object must satisfy ALL properties from both sides
```

	Union (pipe operator)	Intersection (`&`)
Reads as	A **or** B	A **and** B
Value must satisfy	at least one side	every side
Typical use	"number or undefined", literal sets	composing object types from parts
Effect on object shape	fewer guaranteed properties	all properties of all parts

Pitfall: Writing `TPerson | { salary: number }` when you meant "a person *with* a salary". The union lets an object with only `salary` pass; it is `&` that demands every property.

---

interface vs type and Naming Conventions

When to use which

Use case	Use
Shape of an object	`interface`
Shape of a class	`interface`
Union of literal values (e.g. status)	`type`
Union of data types (e.g. "number or undefined")	`type`
Combining multiple types (intersection)	`type`
Anything that is not an object or class	`type`

Naming conventions (expected by leads and managers)

- All **interfaces** are prefixed with `I`: `IProduct`, `IUser`, `IOrder`.
- All **type aliases** are prefixed with `T`: `TStatus`, `TBook`, `TUndefinedNumber`.

These are **not enforced** by TypeScript — your code compiles without them. They make the codebase readable at a glance (you immediately know whether you are looking at an interface or a type alias), and senior developers and tech leads expect them in any professional project.

```
// Correct naming
interface IProduct { /* ... */ }
interface IUser { /* ... */ }
type TStatus = "approved" | "pending" | "rejected";
type TBook = { /* ... */ };

// Usage
// const product: IProduct = { ... };
// const myStatus: TStatus = "approved";
```

General rules to always follow

- All source code lives in `src/` with the `.ts` extension — never `.js` during development.
- **Never deploy TypeScript files** — compile first with `npx tsc`, then deploy from `dist/`.
- Mark TypeScript-related packages as `devDependencies` with `--save-dev`.
- Runtime packages (`express`, `mysql2`, `bcrypt`, `jsonwebtoken`) are always regular `dependencies`.

The next session continues with running TS code, compiling and generating builds.

Combined Worked Example

Two small projects tie the day together: Part A is the router-prefix pattern (JavaScript, runnable with Express); Part B is a TypeScript file using every typing feature from the lecture under the lecture's `tsconfig.json`.

Part A: prefix and middleware in `app.use()`

Setup:

```
mkdir prefix-demo
cd prefix-demo
npm init -y
npm i express
```

Add `"type": "module"` to `package.json`, then create these files.

```
// verify-token.js - stand-in for the Day 9 middleware: only checks the header exists
export function verifyToken(request, response, next) {
  const authHeader = request.get("authorization");
  if (!authHeader) {
    response.status(401).send({ message: "token is missing" });
    return;
  }
  next();
}
```

```
// product-router.js - ONLY suffixes, no "/product" anywhere
import express from "express";

const router = express.Router();

router.get("/", (request, response) => {
  response.send({ message: "all products" });
});

router.get("/:id", (request, response) => {
  response.send({ message: "product " + request.params.id });
});

router.post("/", (request, response) => {
  response.send({ message: "product created" });
```

```
});

router.put("/:id", (request, response) => {
  response.send({ message: "product " + request.params.id + " updated" });
});

router.delete("/:id", (request, response) => {
  response.send({ message: "product " + request.params.id + " deleted" });
});

export default router;
```

```
// admin-router.js - ONLY suffixes, no "/admin" anywhere
import express from "express";

const router = express.Router();

router.get("/", (request, response) => {
  response.send({ message: "all admins" });
});

router.post("/login", (request, response) => {
  response.send({ message: "login endpoint (open)" });
});

export default router;
```

```
// index.js - the prefix and the shared middleware are written ONCE, here
import express from "express";
import productRouter from "./product-router.js";
import adminRouter from "./admin-router.js";
import { verifyToken } from "./verify-token.js";

const app = express();
app.use(express.json());

app.use("/product", verifyToken, productRouter); // guarded
app.use("/admin", adminRouter);                  // open (contains login)

app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

Run `node index.js`.

Trace:

Request	Prefix match	Middleware	Router route	Response
`GET /product` (no header)	`/product`	`verifyToken` → 401	not reached	`token is missing`
`GET /product` + `Authorization: Bearer x`	`/product`	passes, `next()`	`get('/')`	`all products`
`DELETE /product/3` + header	`/product`	passes	`delete('/:id')`	`product 3 deleted`
`POST /admin/login` (no header)	`/admin`	none	`post('/login')`	`login endpoint (open)`
`GET /order`	no prefix matches	—	—	404 (Express default)

Part B: a complete TypeScript project

Setup:

```
mkdir ts-demo
cd ts-demo
npm init -y
npm i typescript tsx @types/node --save-dev
mkdir src
npx tsc --init
```

Add `"type": "module"` to `package.json` and replace the generated `tsconfig.json` with the lecture's version:

```
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "rootDir": "./src",
    "outDir": "./dist",
    "sourceMap": true,
    "noImplicitReturns": true,
    "noImplicitOverride": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true
```

```
  },
  "include": ["src"],
  "exclude": ["node_modules", "dist"]
}
```

Now `src/index.ts`:

```ts
// src/index.ts - Day 10 combined TypeScript example

// ——— Type aliases (T prefix) ———
type TStatus = "approved" | "pending" | "rejected";
type TUndefinedNumber = number | undefined;

type TPerson = {
  name: string;
  phone: string;
  email: string;
};

// Intersection: everything in TPerson AND salary
type TEmployee = TPerson & {
  salary: number;
};

// ——— Interfaces (I prefix) ———
interface IProduct {
  readonly id: number; // set once, never reassigned
  name: string;
  price: number;
  quantity: number;
  description?: string; // optional
}

interface IOrder {
  orderId: number;
  status: TStatus; // only "approved" | "pending" | "rejected"
  product: IProduct;
}

// ——— Typed functions ———
function sum(a: number, b: number): number {
  return a + b;
}

function show(message: string): void {
  console.log(message);
}

function totalValue(list: IProduct[]): number {
  let total: number = 0;
```

```
  for (const p of list) {
    total = sum(total, p.price * p.quantity);
  }
  return total;
}

function findById(list: IProduct[], id: number): IProduct | undefined {
  return list.find((p) => p.id === id);
}

function statusMessage(status: TStatus): string {
  switch (status) {
    case "approved":
      return "Order approved";
    case "pending":
      return "Order is waiting";
    case "rejected":
      return "Order rejected";
  }
}

// ——— Data ———
const productsList: IProduct[] = [
  { id: 101, name: "Keyboard", price: 1500, quantity: 10 },
  { id: 102, name: "Mouse", price: 800, quantity: 25, description: "Wireless" },
  { id: 103, name: "Headphone", price: 2000, quantity: 5 },
];

const emp: TEmployee = {
  name: "Shivam",
  phone: "9999999999",
  email: "shivam@example.com",
  salary: 50000,
};

let discount: TUndefinedNumber = 10;
discount = undefined; // allowed: number | undefined

// ——— Use everything ———
show("Total stock value = " + totalValue(productsList)); // 15000 + 20000 + 10000 =
45000

const found = findById(productsList, 102);
if (found !== undefined) {
  show(found.name + " → " + (found.description ?? "no description"));
}
show("Keyboard description: " + (productsList[0]?.description ?? "no description"));

const order: IOrder = {
  orderId: 1,
  status: "pending",
```

```
  product: productsList[2] as IProduct,
};
show(statusMessage(order.status));

show(emp.name + " earns " + emp.salary);
show("Discount is " + discount);

// ——— The lecture's mistakes, all commented out (uncomment one at a time to see the error) ———
// let x: number = 5; x = "hello";                        // string not assignable to number
// sum(3, "eight");                                       // string not assignable to number
// const bad: IProduct = { id: 1, name: "A", price: 1 };  // quantity is missing
// const first = productsList[2] as IProduct; first.id = 5; // id is read-only
// let s: TStatus = "hello";                              // not one of the three literals
```

Run it on the developer machine, then build for deployment:

```
npx tsx src/index.ts       # dev: compile + run in one step
npx tsc                    # build: writes dist/index.js (+ index.js.map)
node dist/index.js         # what a server would run: plain JavaScript
```

Output:

```
Total stock value = 45000
Mouse → Wireless
Keyboard description: no description
Order is waiting
Shivam earns 50000
Discount is undefined
```

End-to-end trace

1. `npx tsx src/index.ts` — `tsx` type-strips and runs the file in one step (development only).
2. `totalValue(productsList)` loops over three `IProduct` objects: `1500 × 10 = 15000`, `800 × 25 = 20000`, `2000 × 5 = 10000`; `sum` accumulates them to **45000**. Every argument and return value is a `number`, checked at compile time.
3. `findById( … , 102)` returns `IProduct | undefined`. The `if (found !== undefined)` check *narrows* the type so `found.name` is safe. Its `description` is `"Wireless"`.
4. `productsList[0]?.description` — under `noUncheckedIndexedAccess` an array read is `IProduct | undefined`, hence the `?.`. The keyboard has no `description` key, so the read is `undefined` and `??` supplies the fallback text.

5. `IOrder.status` is a `TStatus`, so `"pending"` is accepted and `statusMessage` returns `"Order is waiting"`. The `switch` returns in every branch (`noImplicitReturns`, `noFallthroughCasesInSwitch` satisfied).
6. `TEmployee` is `TPerson & { salary }` — all four properties are present, otherwise the compiler would refuse.
7. `discount` is `number | undefined`, so assigning `undefined` is legal and prints `Discount is undefined`.
8. `npx tsc` reads `rootDir: ./src`, writes `dist/index.js` (`outDir`), skips `node_modules` and `dist` (`exclude`). Only `dist/` would be deployed; the types are gone and no compiler runs on the server.

Quick Revision Checklist

Topic	One-line takeaway
Router prefix	Write `/product`, `/admin` once in `app.use()`; routers define only the suffix.
Public URLs	Moving the prefix does not change any public URL.
Middleware once	`app.use('/product', verifyToken, productRouter)` guards every product route.
Login stays open	Never put `verifyToken` in front of the admin/login router.
Why TypeScript	JS has no type safety — errors surface at runtime; TS adds data types at development time.
TS = JS + types	All JS syntax is valid TS; only type annotations are new.
Never deploy TS	Compile to JS first (`npx tsc`) and deploy `dist/` — runtime compilation wastes time.
`tsc`	The TypeScript compiler (like `javac` for Java), plus the type-checker engine.
`tsx`	Dev-only engine that compiles and runs `.ts` automatically (like nodemon); never in production.
`@types/node`	Type definitions for Node built-ins (`fs`, `http`, `os`, `path`).
`dependencies`	Needed to run: express, mysql2, bcrypt, jsonwebtoken.

Topic	One-line takeaway
`devDependencies`	Dev only: typescript, tsx, @types/node, ESLint; skipped with `npm ci --omit=dev`.
`npx tsc --init`	Creates `tsconfig.json`; plain `tsc` fails because TS is installed locally.
`rootDir` / `outDir`	Source folder (`src`) / compiled output folder (`dist`, the deployable one).
`include` / `exclude`	Compile `src`; never `node_modules` or `dist`.
`noUncheckedIndexedAccess`	Array reads may be `undefined` — compiler warns.
`noUnused…` / `noImplicitReturns`	Unused locals/params and missing returns become errors.
`noFallthroughCasesInSwitch`	A `switch` case without `break` is an error.
`noImplicitOverride`	Overriding a parent method requires the `override` keyword.
Typed variable	`let x: number = 5;` — assigning a string is an error.
Typed function	`function sum(a: number, b: number): number`; `void` = returns nothing.
Typed array	`let arr: number[]`; `IProduct[]` for objects of an interface.
`interface`	Object/class shape; rejects missing and unknown keys; prefix `I`.
Optional `?`	Property may be absent; reading it gives `undefined`.
`exactOptionalPropertyTypes`	Absent optional → `undefined` automatically; writing `undefined` explicitly is an error.
`readonly`	Property cannot be reassigned after creation.
`type` alias	For literal unions, union of types, intersections; prefix `T`.
Literal union	`TStatus` limits a variable to exactly `"approved"`, `"pending"` or `"rejected"` — something an interface cannot do.
Union (pipe operator)	One type **or** another.
Intersection `&`	**All** combined types at once; reuse shared pieces like `TPerson`.
Next session	Running TS code, compiling, generating builds.

Practice Exercises

Concept

1. Why is repeating `/product` on every route inside a router considered error-prone?
2. After moving the prefix into `app.use('/product', ...)`, what does the route `router.get('/:id', ...)` match publicly?
3. Why is `verifyToken` attached to `/product` but **not** to `/admin`?
4. Name three problems of JavaScript's dynamic typing that TypeScript addresses.
5. Why must TypeScript files never be deployed to a server?
6. What are the two things the `typescript` package provides? What is Java's counterpart of `tsc`?
7. Explain the roles of `typescript`, `tsx` and `@types/node`.
8. What decides whether a package goes in `dependencies` or `devDependencies`? Why do cloud deployments care?
9. Why does `tsc --init` fail when TypeScript was installed with `--save-dev`, and what is the fix?
10. What is the difference between `rootDir` and `outDir`, and which folder is deployed?
11. When must you use `type` instead of `interface`? Give the lecture's example.
12. What is the difference between `|` and `&`?

Application

13. Rewrite `router.delete('/order/:id', verifyToken, deleteOrder)` for a router mounted with the prefix in `index.js`, and write the matching `app.use` line.
14. Write the command that installs the three core TypeScript development dependencies.
15. Write a typed function `area` that takes `width` and `height` (numbers) and returns their product as a number.
16. Declare an array `marks` that accepts only numbers, and show the line that would be rejected by `marks.push("A")`.
17. Define an interface `IStudent` with `id` (number), `name` (string), a **read-only** `rollNo` (number) and an **optional** `email` (string).
18. Define `TLevel` that can only hold `"beginner"`, `"intermediate"` or `"advanced"`, and an interface `ICourse` that uses it for a `level` field.
19. Build `TManager` from `TPerson` (`name`, `phone`, `email`) plus `teamSize: number` without repeating the person fields.

20. Write the command a DevOps script would use to install only production dependencies.

Predict output

21. What happens with `let n: number = 10; n = "ten";` in TypeScript? And in JavaScript?
22. Given `function sum(a: number, b: number): number { return a + b; }`, what do `sum(3, 8)` and `sum(3, "eight")` do?
23. Given `description?: string` and an object that omits `description`, what does `product.description` evaluate to?
24. With `exactOptionalPropertyTypes: true`, is `{ id: 1, name: "A", price: 5, quantity: 2, description: undefined }` valid for `IProduct`? What if it is `false`?
25. For `let status: TStatus = "approved"; status = "banana";` — what happens?

Spot the bug

26. Find the problem in this routing code, assuming `index.js` has `app.use('/product', productRouter)`:

```
// product-router.js
router.get('/product', getAllProducts);
router.get('/product/:id', getProductById);
```

27. Find the problems in this TypeScript:

```
interface Product {
  id: number;
  name: string;
  readonly price: number;
}

const p: Product = { id: 1, name: "Pen" };
p.price = 20;
```

28. A student writes `type TEmployee = TPerson | { salary: number };` and expects every employee to have `name`, `phone`, `email` **and** `salary`. What is wrong, and what is the fix?
29. A developer installs `typescript` with plain `npm i typescript`, then deploys the whole project (including `src/*.ts`) and runs it with `tsx` on the server. List what is wrong.
30. With this tsconfig, `tsc` compiles every `.ts` file in the project folder (not only `src`) and the compiled output does not mirror the `src` structure the way you intended. Which settings are missing?

```
{
  "compilerOptions": {
    "outDir": "./dist"
  }
}
```

Practice Answers

1. If the prefix changes, every route in the file has to be edited; missing one creates a bug visible only when someone calls that URL. The same prefix typed many times invites typos.

2. It matches `GET /product/<id>` — e.g. `/product/7`. Express strips the mounted prefix `/product`, so the router sees only `/7` and matches `/:id`.

3. Every product route needs a logged-in admin, so one `verifyToken` in `app.use('/product', verifyToken, productRouter)` covers them all. The admin router contains `POST /admin/login`, which must stay open — otherwise nobody could ever get a token.

4. Any three: variables have no fixed type and silently change type; function arguments have no enforced types (`sum(3, "eight")` gives `"3eight"`); functions have no declared return type; JSON objects have no enforced shape (required keys missing, extra keys appear). All surface only at runtime.

5. The runtime would have to compile `.ts` to JS on every execution (every request, every function call), wasting time. The correct flow is write TS → compile to JS → deploy the JS from `dist/`.

6. The `tsc` compiler (`.ts` → `.js`) and the type-checker engine (raises type errors as you write). Java's counterpart is `javac`.

7. `typescript` — the core package, brings `tsc` and the type checker. `tsx` — dev-only engine that compiles and runs `.ts` automatically (like nodemon for TS); never used in production. `@types/node` — type definitions for Node's built-in modules (`fs`, `http`, `os`, `path`) so TypeScript knows their functions and types.

8. Whether the application needs it to **run**: yes → `dependencies` (express, mysql2, bcrypt, jsonwebtoken); only for development → `devDependencies` (typescript, tsx, @types/node, ESLint). `node_modules` is heavy and cloud servers charge per byte, so deployment scripts install only dependencies (`npm ci --omit=dev`).

9. `--save-dev` installs TypeScript locally in the project's `node_modules`. Plain `tsc` searches the global location, where it does not exist, so the shell reports it as not recognised. Use `npx tsc --init`, which looks in the project's `node_modules`.

10. `rootDir` is where the TypeScript source is read from (`./src`) and affects whether the folder structure is preserved; `outDir` is where compiled JavaScript is written (`./dist`). The `dist` folder is deployed; `src` never is.

11. When the type is not an object/class shape — for example a union of literal values. `type TStatus = "approved" | "pending" | "rejected"` restricts a variable to those three strings; an interface cannot do this.

12. `|` (union) means one type **or** another — a value needs to satisfy only one side. `&` (intersection) means **all** combined types at once — a value must have every property from every part.

13.

```
// order-router.js
router.delete('/:id', deleteOrder);

// index.js
app.use('/order', verifyToken, orderRouter);
```

14.

```
npm i typescript tsx @types/node --save-dev
```

15.

```
function area(width: number, height: number): number {
  return width * height;
}
```

16.

```
let marks: number[] = [80, 90];
marks.push("A");   // Error: Argument of type 'string' is not assignable to parameter
of type 'number'
```

17.

```
interface IStudent {
  id: number;
  name: string;
  readonly rollNo: number;
  email?: string;
}
```

18.

```ts
type TLevel = "beginner" | "intermediate" | "advanced";

interface ICourse {
  id: number;
  title: string;
  level: TLevel;
}
```

19.

```ts
type TManager = TPerson & {
  teamSize: number;
};
```

20.

```
npm ci --omit=dev
```

21. TypeScript: error — `Type 'string' is not assignable to type 'number'`, caught before the code runs. JavaScript: no error, `n` silently becomes a string.

22. `sum(3, 8)` returns `11`. `sum(3, "eight")` is a compile-time error: `Argument of type 'string' is not assignable to parameter of type 'number'`. (In plain JS it would silently return `"3eight"`.)

23. `undefined`. No error is raised; an absent optional property reads as `undefined`.

24. With `exactOptionalPropertyTypes: true` it is **invalid** — `Type 'undefined' is not assignable to type 'string'`; simply omit the key. With `false` the error disappears and the object is valid.

25. Compile-time error: `Type '"banana"' is not assignable to type 'TStatus'`. Only `"approved"`, `"pending"` and `"rejected"` are accepted (a plain `string` variable would have accepted `"banana"`).

26. The prefix is repeated: the public URLs become `/product/product` and `/product/product/:id`, so `GET /product` returns 404. The router should define only the suffixes: `router.get('/', ...)` and `router.get('/:id', ...)`.

27. (a) The object literal is **missing** `price`, a required property → error (`readonly` does not make a property optional). (b) `p.price = 20` is not allowed because `price` is read-only. (c) Naming convention: the interface should be `IProduct`, not `Product`.

28. `|` is a union: an object only needs to satisfy **one** side, so `{ salary: 5 }` alone passes. Use the intersection: `type TEmployee = TPerson & { salary: number };`.

29. (a) `typescript` should be installed with `--save-dev` (it is a devDependency). (b) TS must never be deployed — compile with `npx tsc` and deploy `dist/` only. (c) `tsx` is dev-only and must not run in production; it compiles on every execution. The server should run `node dist/index.js`.

30. `rootDir` is missing (the compiler guesses the common root, so the output structure may differ from what you expect) and `include` is not set (so every `.ts` file under the project folder is compiled). Add `"rootDir": "./src"` and `"include": ["src"]`; also add the explicit `"exclude": ["node_modules", "dist"]` as the lecture does so third-party packages and old output are never compiled.